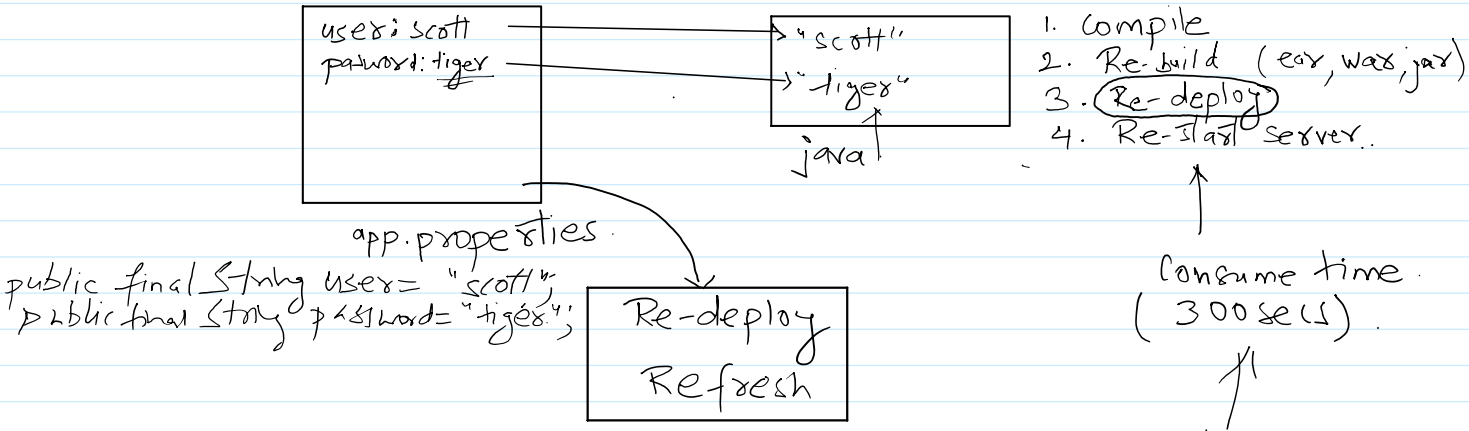
Properties

31 May 2025 20:20



In our program, if anything which changes frequently (like user name, passwords, mail ids, mobile number, etc.) Are not recommended to hard code (keeping the values directly) in the Java program. Because if there is any change to reflect that change, recompilation, rebuild, redeploy applications are required even sometimes server restart also required, which creates business impact to the client.

OTP →

We can overcome this problem using properties file in such type of variable things we have to configure in properties file that, we have to read into Java program and we can use those properties. The main advantage of this approach is if there is a change in properties file to reflect that change, just redeployment is enough, which won't create any business impact to the client.

We can use Java properties object to hold properties which are coming from properties file.

```
HashMap m = new HashMap();
Properties p = new Properties();
```

In normal Map(like HashMap, Hashtable, TreeMap) key-value can be any type. But in case of Properties key-value should be String type.

Methods:

String setProperty(String pname, String pvalue);

(old value) If the specified property already available, then old value will be replaced with new value and returns the old value.

This function is used to set a new property.

String getProperty(String pname);

To get the value associated with the specified property. If the specified property is not available then this method returns 'null'.

Enumeration propertyNames();

returns all property names present in properties object.

void load(InputStream is);

void load(InputStream is);

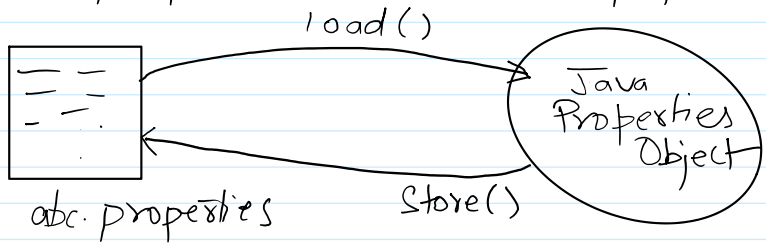
↑

to load properties from properties file into Java properties object.

void store(OutputStream os);

↑

to store properties from Java properties object into properties file.




```
package Chapter5;
import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.FileOutputStream;
import java.io.IOException;
import java.util.Properties;
import javax.swing.JOptionPane;
public class PropertiesDemo {
   public static void main(String[] args) {
      Properties p = new Properties();
      try (FileInputStream fis = new FileInputStream("Chapter5/users.properties");
      FileOutputStream fos = new FileOutputStream("Chapter5/users.properties", true)){
         p.load(fis);
         String str  = p.getProperty("Subhalakshmi");
         System.out.println("\nvalue: "+str);
         String uname = JOptionPane.showInputDialog("Enter user name: ");
         String pwd = JOptionPane.showInputDialog("Enter your password: ");
         // p.setProperty(uname, pwd);
         p.store(fos, uname+"="+pwd);
      } catch (FileNotFoundException e) {
         // TODO Auto-generated catch block
         e.printStackTrace();
      } catch (IOException e) {
         // TODO Auto-generated catch block
         e.printStackTrace();
      }
   }
}
```

Output:

```
value: 1234@0987
```

```
Subhalakshmi=1234@0987
Aadit=0987@1234#shiv=123@456
#Sun Jun 01 10:17:00 IST 2025
Aadit=0987@1234
Subhalakshmi=1234@0987
```

Official Documentation

public class Properties extends Hashtable<Object,Object>

The Properties class represents a persistent set of properties. The Properties can be saved to a stream or loaded from a stream. Each key and its corresponding value in the property list is a string.
A property list can contain another property list as its "defaults"; this second property list is searched if the property key is not found in the original property list.
Because Properties inherits from Hashtable, the put and putAll methods can be applied to a Properties object. Their use is strongly discouraged as they allow the caller to insert entries whose keys or values are not Strings. The setProperty method should be used instead. If the store or save method is called on a "compromised" Properties object that contains a non-String key or value, the call will fail. Similarly, the call to the propertyNames or list method will fail if it is called on a "compromised" Properties object that contains a non-String key.
The iterators returned by the iterator method of this class's "collection views" (that is, entrySet(), keySet(), and values()) may not fail-fast (unlike the Hashtable implementation). These iterators are guaranteed to traverse elements as they existed upon construction exactly once, and may (but are not guaranteed to) reflect any modifications subsequent to construction.
The load(Reader) / store(Writer, String) methods load and store properties from and to a character based stream in a simple line-oriented format specified below. The load(InputStream) / store(OutputStream, String) methods work the same way as the load(Reader)/store(Writer, String) pair, except the input/output stream is encoded in ISO 8859-1 character encoding. Characters that cannot be directly represented in this encoding can be written using Unicode escapes as defined in section 3.3 of The Java Language Specification; only a single 'u' character is allowed in an escape sequence.
The loadFromXML(InputStream) and storeToXML(OutputStream, String, String) methods load and store properties in a simple XML format. By default the UTF-8 character encoding is used, however a specific encoding may be specified if required. Implementations are required to support UTF-8 and UTF-16 and may support other encodings. An XML properties document has the following DOCTYPE declaration:
<!DOCTYPE properties SYSTEM "http://java.sun.com/dtd/properties.dtd">

Note that the system URI (http://java.sun.com/dtd/properties.dtd) is *not* accessed when exporting or importing properties; it merely serves as a string to uniquely identify the DTD, which is:

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- DTD for properties -->
<!ELEMENT properties ( comment?, entry* ) >
<!ATTLIST properties version CDATA #FIXED "1.0">
<!ELEMENT comment (#PCDATA) >
<!ELEMENT entry (#PCDATA) >
<!ATTLIST entry key CDATA #REQUIRED>
```

This class is thread-safe: multiple threads can share a single Properties object without the need for external synchronization.

API Note:

The Properties class does not inherit the concept of a load factor from its superclass, Hashtable.

Since:

1.0

From <https://docs.oracle.com/en/java/javase/24/docs/api/java.base/java/util/Properties.html>